Urban Eye AI-Driven Smart Complaint & Civic Issue Resolution System

Alok Ranjan
Department of Information Technology
Guru Tegh Bahadur Institute of Technology
New Delhi, India
alokranjann2005@gmail.com

***Abstract*— UrbanEye is a civic complaint platform developed to make issue reporting easier for residents and more usable for administrators. The system brings together a web interface, a mobile application, a Node.js and Express backend, and a FastAPI AI service. Instead of saving a complaint only as a written message, UrbanEye enriches it with category, priority, likely department, suggested action, confidence, and a draft update suitable for X-style public communication. These extra fields make the complaint easier to review, filter, and track. The present implementation gives priority to text-based analysis, dashboard visibility, and dependable fallback behavior for local testing and project demonstration. Overall, the project shows how unstructured civic complaints can be converted into organized records that support clearer tracking and more transparent grievance handling.**

***Index Terms*— civic technology, AI-assisted triage, complaint management, smart governance, multi-platform systems, public grievance redressal, X-ready post generation, web and mobile integration, agent-oriented workflow, e-governance.**

I. INTRODUCTION

City services increasingly use digital platforms, but the complaint process is still difficult for many residents. A person may notice garbage overflow, a damaged road, a streetlight failure, or a water leak, yet the reporting path can feel slow and confusing. Existing systems often expect users to select departments, fill detailed forms, and then wait with limited feedback. As a result, the act of filing a complaint may not give citizens enough confidence that the issue is being handled.

UrbanEye addresses this gap through a practical reporting and tracking workflow. Its purpose is to keep complaint submission simple while making the record more structured after it enters the system. A complaint can be created from the web or mobile application, processed by the backend, analyzed by the AI service, and then displayed through dashboard and detail screens. The same flow also prepares a short X-style update that may later support supervised public communication.

A key motivation behind the project is that many systems only collect complaints and do little to prepare them for action. UrbanEye treats submission as the first step in a larger workflow. The AI layer studies the complaint text to infer the issue type, possible urgency, suitable department, and an initial response suggestion. This does not remove the need for human review, but it gives reviewers a cleaner starting point than raw text alone.

The project was designed as a modular application rather than a single-screen prototype. The web interface supports overview and administrative visibility, while the mobile interface focuses on quick reporting and personal tracking. The backend manages validation, APIs, and data flow. The AI service is kept separate so that complaint analysis can evolve independently. This separation makes the system easier to test, explain, and improve.

This paper presents the implemented project and explains how its parts work together. The discussion focuses on system architecture, complaint flow, AI-assisted triage, storage, dashboards, and the practical limits of the current version.

II. LITERATURE REVIEW

A. Digital Grievance Redressal and Civic Technology

Digital governance has improved access to many public services, but grievance systems still face a common weakness. They may record a complaint successfully without giving the citizen a clear idea of what happens next. If routing is unclear and status updates are limited, the platform can feel passive even when the complaint has been logged. Effective civic systems therefore need both proper registration and visible follow-up.

B. AI for Complaint Interpretation

Artificial intelligence is useful in complaint handling because residents usually describe problems in informal language. Their reports may include landmarks, short phrases, mixed wording, or emotional descriptions rather than official categories. Text analysis can convert this type of input into structured fields such as issue type, urgency, and probable department.

This conversion matters because the language used by citizens and departments is often different. A resident may write that dirty water is spreading in a lane, while an administrative system may need to treat it as a drainage or water-supply issue. AI-assisted interpretation helps reduce this gap between everyday reporting and official handling.

C. Agent-Oriented System Design

Modern applications are easier to maintain when major responsibilities are separated. UrbanEye follows this approach by keeping intake, AI analysis, storage, dashboard display, and message drafting as distinct parts of the system. This structure keeps the current project manageable and also leaves room for later upgrades.

D. Public Accountability and Social Communication

Public platforms such as X are frequently used to raise civic issues because they can create visibility. However, posts written directly by users may miss useful details or may use unclear wording. UrbanEye therefore creates a short factual draft from the complaint record itself. Direct posting is not included in the current version and is reserved for supervised future work.

This design keeps public communication cautious. The generated text can help prepare a clearer update, but a human can still review it before it is shared outside the system.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

A. Overall Architecture

UrbanEye is organized around four connected parts: a web application, a mobile application, a backend API, and an AI service. The web app supports dashboards and broader visibility. The mobile app supports quick complaint creation and personal tracking. The backend coordinates data flow and complaint creation, while the AI service analyzes complaint text and returns structured information.

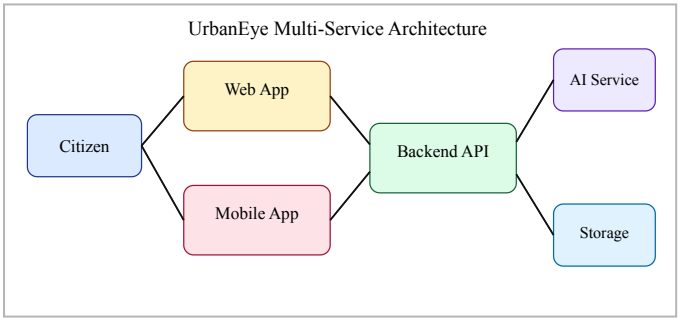

Fig. 1. High-level architecture of the UrbanEye system

B. Complaint Intake and AI-Assisted Triage

The workflow begins when a user submits a complaint with a title, description, and location. When available, media and coordinates may also be included. The backend validates the request and forwards the complaint text to the AI service. In the current version, the service uses rule-based text analysis with structured response generation. It returns category, priority, department, sentiment, confidence, suggested action, and a public-facing draft post.

This design was selected for stability during implementation and demonstration. It does not depend completely on a generative model, yet it still shows how automated interpretation can improve a civic workflow. Complaints related to sanitation, electricity, water, roads, and general issues are mapped to likely departments, and safety-related terms can raise the priority level.

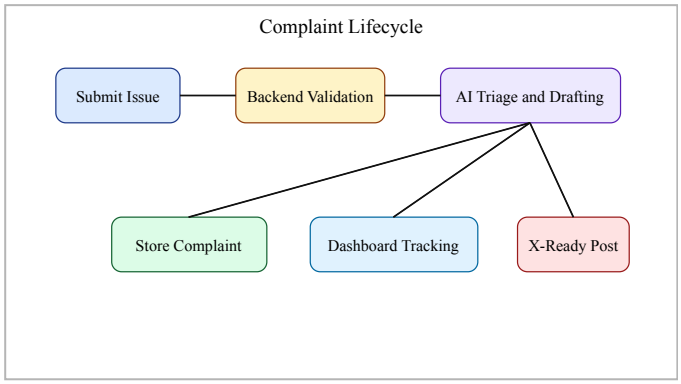

Fig. 2. End-to-end workflow from complaint submission to monitoring and public-message drafting

C. Persistence and Complaint State

The data layer is designed around PostgreSQL with Prisma and stores both user and complaint records. Each complaint keeps the original user input along with status, category, department, sentiment, confidence, suggested action, social post, coordinates, and image reference fields. The system also includes fallback behavior, especially on the mobile side, so that the user experience can continue when every service is not available at the same time.

This fallback behavior is important in a project environment where one service may be running while another is temporarily unavailable. By supporting local continuity and fallback flows, UrbanEye remains usable during testing and demonstration. It also follows an important engineering principle for public-facing systems: failures should be handled gracefully.

D. User-Facing Tracking and Dashboarding

A major goal of UrbanEye is to make complaint progress easier to understand. For that reason, the dashboard does more than list records. It shows whether complaints are pending, in progress, or resolved, and it includes summary counts that quickly describe the current situation. On mobile, this appears as a personal tracking view. On the web, it appears as a wider dashboard with cards and summary sections.

The difference between the web and mobile experience is intentional. The web view is more useful for overview and monitoring, while the mobile view is better for quick reporting and simple follow-up. The system therefore adapts the presentation to the way each platform is likely to be used.

E. Human-in-the-Loop Control

Although the system adds AI-assisted analysis, final responsibility remains with human reviewers. A suggested department, priority level, or drafted X post is meant to support decisions rather than replace them. This is necessary because real civic issues can be unclear and may require local context that an automated system does not have.

F. Status Model and Resolution Semantics

Complaint tracking in UrbanEye uses a simple status model. Each complaint is marked as pending, in progress, or resolved, and the same status appears across cards, summaries, and detail views. Although the model is basic, it helps users see that a complaint remains part of a visible process after submission.

TABLE I
COMPLAINT STATUS SEMANTICS IN URBANEYE

Status	Meaning	User Interpretation
Pending	Complaint has been created and awaits action	The issue is registered but work has not visibly started
In Progress	Complaint has entered an active handling phase	The issue is being reviewed or worked upon
Resolved	Complaint has been marked complete	The issue is considered solved from the system perspective

TABLE II
CORE MODULES OF THE PROPOSED SYSTEM

Module	Primary Function	Current Role in System
Web Interface	Dashboard, reporting, complaint visibility	Provides broad operational overview and reporting flow
Mobile Interface	Complaint creation, personal tracking, profile settings	Improves resident-side ease of use and continuity
Backend API	Validation, complaint creation, aggregation, normalization	Connects clients, storage, and AI service
AI Service	Category, priority, department, suggestion, social-post output	Enriches complaints with structured triage metadata
Persistence Layer	User and complaint data storage	Supports PostgreSQL-ready and fallback operation

IV. IMPLEMENTATION DETAILS AND ANALYTICAL MODEL

A. Complaint Classification Logic

The current AI workflow uses structured text analysis to estimate the most likely complaint category. Terms related to garbage and waste are mapped to sanitation, streetlight and power terms are mapped to electricity, water and leakage terms are mapped to water issues, and pothole or drainage terms are mapped to road-related issues. The method is simple, but it gives the system a stable starting point for triage.

This approach fits the project because it is transparent and easy to verify. Unlike a black-box model, the logic can be inspected directly. A reviewer can understand why a complaint was placed in a category or why certain words increased its priority, which makes the system easier to explain and improve.

B. Priority Estimation

In the current version, priority is assigned through clear rules rather than a trained statistical model. Regular infrastructure issues are marked as medium or high priority depending on the issue type, while safety-related words such as *fire*, *flood*, *accident*, or *unsafe* can raise the complaint to a critical level. This resembles first-level screening by a human operator.

For future deployments, the same logic can be generalized into a weighted priority function:

$$P = (w_s \times S) + (w_u \times U) + (w_r \times R)$$

where S denotes issue severity, U denotes urgency cues in the complaint, and R denotes recurrence or local concentration. Such a formula would make the system more adaptive while preserving the operational logic used in the present rule-based design.

C. X-Ready Social Post Generation

One useful feature of the system is its ability to prepare a short civic update for X-style public communication. The message includes the issue, location, likely department, and relevant hashtags while staying concise. If a language model is available, the service can create a more natural message; otherwise, it falls back to a fixed template. This keeps the output dependable during local use.

The message style is intentionally cautious. It avoids dramatic claims, does not suggest that action has already occurred, and remains based on the complaint data. This is important for reducing misinformation, privacy risk, and misleading public communication.

D. Data Elements Returned by the AI Service

TABLE III
STRUCTURED OUTPUT FIELDS PRODUCED DURING COMPLAINT ANALYSIS

Field	Purpose	User or System Benefit
Category	Classifies the issue type	Supports routing, filtering, and dashboard breakdowns

Field	Purpose	User or System Benefit
Priority	Estimates urgency level	Helps highlight issues that need faster attention
Department	Suggests likely authority	Reduces ambiguity in administrative routing
Sentiment	Captures urgency tone such as concerned or urgent	Improves interpretive context in UI
Suggested Action	Provides a first operational response hint	Supports triage review and field validation
Social Post	Produces a concise public-facing summary	Supports future supervised communication

Example generated complaint summary:
High-priority civic alert: Overflowing garbage near Sector 8 Market. Garbage has not been collected for three days. Assigned to Sanitation Department. #CleanStreets #CivicAction

V. RESULTS AND DISCUSSION

A. Functional Evaluation

Since UrbanEye is a working academic project rather than a city-wide deployment, the evaluation focuses on whether the main workflow operates correctly and consistently. The system accepts complaints, returns structured metadata, stores complaint records, and shows complaint status through dashboards. This is important because the project demonstrates an end-to-end flow instead of a set of isolated screens.

B. Representative Complaint Outcomes

TABLE IV
REPRESENTATIVE COMPLAINT INTERPRETATION OUTPUTS

Complaint Theme	Predicted Category	Priority	Likely Department
Garbage accumulation near market	Sanitation	High	Sanitation Department
Streetlight outage near school crossing	Electricity	Medium to Critical when safety cues appear	Electricity Department
Continuous pipeline leakage	Water	High	Water Department
Pothole and drainage obstruction	Roads	High	Road Department

These examples show that even a simple triage pipeline can make complaint records more useful. Instead of storing only raw text, UrbanEye returns a richer record that can be displayed clearly in the interface. This improves dashboard summaries, prioritization, and user tracking.

Another benefit is platform consistency. Because the backend returns a normalized complaint structure, both the web and mobile applications can use the same fields with limited extra logic. This reduces duplication and keeps the user experience more consistent.

C. Improvement over Conventional Workflows

TABLE V
COMPARISON WITH A CONVENTIONAL MANUAL COMPLAINT FLOW

Criterion	Conventional Flow	UrbanEye Approach
Complaint entry	Mostly form-driven and static	Web and mobile complaint flow with structured enrichment
Initial routing	Human interpretation required	AI-assisted category and department mapping
Status visibility	Often unclear to the user	Dashboard and detail views show complaint state clearly
Public communication support	Manual drafting required	X-ready concise post generated automatically
Resilience during service failure	Commonly brittle	Fallback logic maintains continuity for demos and local use

The main strength of the project is not a single model or screen. It is the connection between complaint intake, AI analysis, storage, tracking, and communication support. This makes UrbanEye more complete than a UI prototype or a separate AI demonstration.

From a software engineering point of view, the project also shows the value of interoperability. The backend can support more than one storage mode, the AI layer is not limited to one model source, and the mobile app can continue locally when connectivity is weak. These choices make the project easier to maintain and more realistic for future extension.

D. Qualitative Observations

During project validation, three practical advantages became clear. First, structured triage reduced ambiguity in complaint records. Second, the dashboard made complaint status easier to understand at a glance. Third, the X-ready message showed how a complaint could be converted into a public update without asking the user to draft it manually.

These observations suggest that useful improvement is possible even before advanced prediction models are added. Better complaint systems depend not only on stronger AI, but also on clearer workflows and better user understanding.

VI. PRACTICAL IMPLICATIONS AND LIMITATIONS

A. Practical Implications

For citizens, the system reduces uncertainty. They do not need to guess the correct department or wait without context after filing a complaint. For administrators, the main benefit is that complaints arrive in a structured form and are easier to summarize. The generated public update also connects internal complaint records with outward-facing accountability.

The project is useful in situations where complaint reporting is frequent but system feedback is weak. Even if the full municipal response process is not automated, clearer status information and complaint history can improve trust. In that sense, transparency becomes an important service outcome.

B. Socio-Technical and Ethical Considerations

Any civic complaint system may handle location data, photos, and sensitive descriptions. This creates privacy and governance concerns. Public communication features must therefore be designed carefully so that user data is not exposed and automated summaries do not misrepresent the complaint. UrbanEye's current focus on X-ready drafting instead of direct posting supports that caution.

Bias is another important issue. People describe problems differently depending on language, education, and local vocabulary. A narrow classifier may work well for some users and less well for others. For that reason, UrbanEye is presented as an evolving platform, not a final production system. Wider deployment would require multilingual testing, more diverse data, and stronger review workflows.

C. Limitations

The current implementation has clear limits. First, the active triage flow still depends mainly on rule-based text analysis rather than a trained multimodal model. Second, image upload is supported, but full image-based issue detection is not part of the deployed pipeline yet. Third, the system can create X-ready text, but it does not post directly through the X API. Finally, the project has not been tested at municipal scale, so it should be understood as a strong prototype rather than a production platform.

The confidence value returned by the current AI service should also be interpreted carefully. In a mature machine-learning system, confidence would normally come from training data and benchmark testing. In this project, it mainly acts as an interface signal rather than a fully validated reliability measure.

VII. CONCLUSION

UrbanEye shows that civic complaint systems can be improved meaningfully without requiring a full enterprise smart-city setup at the start. By combining web and mobile reporting with a structured backend and a dedicated AI service, the project turns plain complaint text into clearer, more useful, and status-aware records. It also improves transparency by showing users whether an issue is pending, in progress, or resolved.

The value of the project lies not only in using AI, but also in designing a complete workflow. Complaint intake, analysis, storage, dashboarding, and public-message drafting are connected clearly instead of being treated as separate features. Because of this, UrbanEye works as a practical civic-tech prototype and shows how AI can support urban governance when it is built into a complete complaint-management process.

VIII. FUTURE SCOPE

Several future improvements can make the system stronger. The first is a computer-vision module for image-based issue detection. The second is supervised X API integration so that approved complaint updates can be posted directly. Another direction is richer agent-based orchestration, where separate vision, routing, and communication agents work together more deeply. Other useful extensions include multilingual complaint handling, geospatial hotspot analysis, notifications, and closer integration with official civic dashboards.

Another promising improvement is smarter priority assignment based on complaint density, repeated reports, and department workload. Instead of relying only on wording, the system could consider how often similar issues appear in the same area and how quickly they are usually resolved. This would move UrbanEye closer to an intelligent decision-support system for city administration teams.

IX. REFERENCES

[1] Government of India, “Centralized Public Grievance Redress and Monitoring System (CPGRAMS),” DARPG. [Online]. Available: https://pgportal.gov.in/

[2] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. John Wiley & Sons, 2009.

[3] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, 2017.

[4] T. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems*, 2020.

[5] FastAPI, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com/

[6] Prisma, “Prisma ORM Documentation.” [Online]. Available: https://www.prisma.io/docs

[7] Vercel, “Next.js Documentation.” [Online]. Available: https://nextjs.org/docs

[8] Expo, “Expo Documentation.” [Online]. Available: https://docs.expo.dev/

[9] React Native, “React Native Documentation.” [Online]. Available: https://reactnative.dev/docs/getting-started

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. CVPR*, 2016.

[11] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, 2020.

[12] Ministry of Housing and Urban Affairs, Government of India, “Smart Cities Mission.” [Online]. Available: https://smartcities.gov.in/